The Model Fidelity Hierarchy: From Text to Conceptual, Computational, and Executable Model

Natali Levi, Ahmad Jbara, and Dov Dori (F'17)

Abstract—Model-based systems engineering (MBSE) applies a variety of model kinds, each with its own fidelity and exactness level. Based on experience we gained while modeling an aircraft landing gear with the objective of numerically defining its various parameters that fulfill engineering and safety requirements, we present the model fidelity hierarchy. At this hierarchy's bottom, vaguest level, is spoken language, followed by free written text, conceptual model, its augmentation with computational capabilities, and finally an executable version of that model. Using Object-Process Methodology (OPM ISO 19450) with its computational extension, we present this hierarchy by describing the landing gear model as it progresses through these levels, and the kinds of mistakes revealed while transitioning from one level to the next. The model fidelity hierarchy, identified and defined in this research, is made possible by using OPM, which enables these level transitions to be information lossless, providing the most value while requiring the minimal effort. The ability of this continuous, seamless modeling approach to detect errors with increasing accuracy justifies our OPM approach, as errors revealed in this early system lifecycle stage are exponentially less costly to correct than those revealed downstream.

Index Terms—System modeling, model execution, Model Fidelity Hierarchy, error detection, model-based systems engineering, software engineering, landing gear

Abbreviations

MBSE	Model-based systems engineering
OPM	Object-Process Methodology
OPD	Object-Process Diagram
OPL	Object-Process Language
MAXIM	Methodical Approach to Executable Integrative Modeling
INCOSE	The International Council on Systems Engineering
OOSEM	Object-Oriented System Engineering Method
SysML	Systems Modeling Language
UML	Unified Modeling Language
fUML	Foundational UML
xUML	Executable UML
STKH	Stakeholder
ISO	International Organization for Standardization

1. INTRODUCTION

Model-based systems engineering (MBSE), often defined as the formalized application of modeling to support system requirements, design, analysis, verification, and validation activities, advocates that formal models are the authoritative set of artifacts that anchor product and system development processes and evolve to reflect the evolution of the design. Design usually starts with the most obvious and simple way by writing a specification document [1]. In this step, a designer and a field expert or a product client are involved. They often start with eliciting verbally. This is a major source of miscommunication, since often the parties mean different things when they use the same words. A slight improvement is achieved when the spoken words are put into a formal written document, as putting requirements in writing usually requires more discipline, especially when authoring a binding legal document. Yet, a major problem with such requirements specification documents is that they almost inevitably prone to containing potential mistakes, because there is no automated verification that the document can be passed through, and the writer may write a wrong fact or even two opposite, contradicting facts or requirements in two different places in the same document [2]. It is likely that a reader of a sufficiently long requirements document written in free text would encounter difficulties remembering all the facts and spot all the contradictions within it [3].

Given these severe drawbacks of a free text document, the next, more formal way to design a system is to adopt an MBSE approach – creating a conceptual model of a system. The model is expected to be based on facts and requirements from the specification document and therefore to faithfully reflect what needs to be designed and implemented. By using a proper modeling methodology, language, and tool, some of the contradictions, deficiencies, and redundancies in the free text may be resolved during the modeling process. However, there is a limit to what qualitative conceptual modeling can resolve in terms of overcoming the above shortcomings of free text documents.

We present the model fidelity hierarchy, which includes two new stages: computational modeling and execution. These stages advance the state-of-the-art in ensuring accurate design that meets as many of the original requirements as possible. Each level in the MFH helps to improve the model achieved at the previous level, making it more accurate, as mistakes are detected and eradicated from the model. The overall result is that the system represented by the model becomes less error-prone, which contributes to increasing its soundness. Modeling mistakes can be discovered and fixed by Methodical Approach to Executable Integrative Modeling – MAXIM. MAXIM is an integrated ISO 19450 OPM [4], [5] model-based systems engineering and software engineering methodology. MAXIM is developed and implemented as part

of OPCloud[1] [6] and MAXIM enables the combination of both qualitative (conceptual) and quantitative (computation) elements in the same model, including an execution of the model.

As the case-in-point for this research, we used MAXIM to model a conceptual Vee model of an aircraft's landing gear system and compute its design parameters. In our previous work [7], where we performed model-based requirements analysis, validation, and verification for civil transport aircraft design with dynamic landing constraints, we have shown seamless transition between two stages: MBSE and model-based design (MBD). The initial motivation for this research was to create an integrative conceptual-computational "Vee" model. During the current, follow-up study, we came across the significant and interest general findings regarding model fidelity hierarchy. In view of this, we made a decision to focus on the model fidelity hierarchy while describing the originally planned research. At the upper levels in our previous work, the model was purely conceptual and qualitative. As we drilled down into the details, we started to populate the model with quantitative elements (objects, processes, and relations among them) as the needs arose. However, the computational part in our previous work, which used Matlab, was not integrated into the model. In contrast, in this work we have used MAXIM to expand the original qualitative model with computational and executable capabilities that are embedded into and spread across the originally conceptual model. With MAXIM, these capabilities are added naturally, effortlessly, and seamlessly. The result is a unified model that combines qualitative and quantitative aspects of the landing gear system. The uniqueness of this approach is that it enables one to numerically calculate the various landing gear design parameters as computations that are embedded naturally in their wider qualitative context, rather than as separate, disconnected mathematical equations. While in our previous work [6] the Vee model, the cargo system, and the associated computations were three separate models, in this research, this complex system is represented in a single unifying, seamless model, which includes the three previously separate models. Our approach contributes developing of systems of system as different systems that collaborate to produce a global behavior have more chance for mistakes than when modeling one system. Our approach helps detecting that as all systems will be modeled using the same language constructs using the same environment without any relation to the inherent differences between these systems. Moreover, as OPM has an inherent mechanism for dealing with complexities we expect that modeling complex systems (SoS) is feasible.

One of the new traits we discovered as we were engaged in the new modeling approach modeling is that by adding units, values and computational functions to the model, we detected and corrected inaccuracies in the conceptual model of the aircraft's landing gear. Not less importantly, we were able to find mistakes in the calculations and correct them. In the next, top level of the model fidelity hierarchy, model execution, we were able to verify whether the new model is correct. At this stage too, we discovered additional conceptual and computational mistakes.

The landing gear case study presented in this research demonstrates the importance of our integrated qualitative-quantitative-executable modeling approach, which has given rise to the model fidelity hierarchy as a key, unplanned and unexpected by-product. More specifically, it seems that synergy between the different aspects of the landing gear OPM model yields better results and helps detect flaws when developed as a whole rather than in separate, disjoint consecutive parts. Importantly, the continuum of the levels in the model fidelity hierarchy enables us to trace engineering decisions to business requirements, enabling us to convey the "big picture". For example, the high-level business requirements of transporting goods through air percolates downstream through the model layers till it reaches purely technical details, such as the need to calculate the values of the Airplane's Wheel Nominal Diameter and Tire Nominal Diameter.

Our takeaway message from the research presented here is that a model fidelity hierarchy clearly exists. At its bottom is verbal and written free text documentation, followed by a qualitative, conceptual model, then a combination of qualitative and quantitative model, and ultimately, at the top of the hierarchy, at least currently, is the execution of the integrated qualitative-quantitative model, which provides for sound verification of the model and the attainment of reliable quantitative model parameters.

The contribution of our research is indicated through three dimensions: (1) Theoretical: we added two new stages to the model fidelity hierarchy: computational modeling and execution. As a result, we get a combined hardware-software-humans model that constitutes a complete and accurate executable specification of the system from its top-level abstract view all the way to the most detailed view. (2) Methodological: we presented a methodical approach to modeling mistakes discovering using MAXIM. We represent a method to describe the structure and behavior of the system, as well as its quantitative requirements and computational, software-driven aspects. (3) Practical: OPCloud tool development to support MAXIM usage.

While it may seem obvious that adding modeling efforts to a development process should at the least create additional fidelity in the design, in practice such additions require transitioning from one modeling framework to another one, which has the capabilities that are required in the next stage. These transitions are costly in terms of the effort needed to translate the system model from one framework to the next in line, and it inevitably causes traceability problems and loss of information that is critical to the development process. The originality of our contribution lies in providing the capability to advance from one fidelity level to the next without the need to make these harmful transitions, as all the development

[1] https://www.opcloud.tech/

effort from the requirements model all the way to the computational executable model is done within the same OPM modeling framework, whose capabilities are continually expanded while clinging to the same basic principles of processes that transform stateful objects.

The remainder of this paper is structured as follows. In Section 2, we survey related work. In Section 3, we present MAXIM, which includes OPM and its computational extension, as well as the OPCloud OPM modeling environment. In Section 4, we describe the conceptual OPM landing gear Vee model. In Section 5, we present the upper hierarchy levels and the corresponding error detection stages. In Section 6 we present the final error detection stage which is a model execution. In Section 7 we summarize and discuss our work. Finally, in Section 8, we show the limitation of our research and suggest future research directions.

2. Related work

According to the international standard ISO 15288, a system engineering process can be divided into requirements development, architectural design, technical evaluation, and synthesis [8]. The International Council on Systems Engineering (INCOSE) has adopted the Vee model to define and develop arbitrary systems. The left leg of the Vee model represents the decomposition of requirements and creation of system specifications. The right leg represents integration of parts and validation of the system [7].

SysML, the OMG systems modeling language, is intended to facilitate the application of the modeling approach to create a model of the system [9]. Most SysML models are created for documentation purposes, while others enable better understanding of the underlying system and validation of desirable behavior of that system. In the former, the focus is on syntax and notation, while in the latter, the focus is on execution semantics [10]. Executable models require an engine that “understands” the execution semantics. The purpose of such a simulating engine is to better understand the system and validate its behavior without the need to have access to the real system [11]. The Object-Oriented Systems Engineering Method (OOSEM) is an integrated framework that combines object-oriented techniques, a model-based design approach, and traditional top-down systems engineering practices [12]. The Executable Systems Engineering Method (ESEM) augments OOSEM by enabling executable SysML models [13].

Almost every modern system is a complex combination of hardware and software modules that are meant to provide value to the system beneficiaries. This relationship implies that they should be integrated [11], [12]. One of the conclusions is that the content of a system or different sub-systems in a complex system should be modeled jointly so they are interoperable, well organized, and consistent. However, this goal is hard to achieve [14]. Several solutions to the interoperability problem have been proposed, including levels of conceptual interoperability [15] and openMBEE, which serves as a first step of capturing consistent system data [16]. However, these efforts ended at the conceptual model level, without software integration and model execution. The state of the practice is distant from the ideal. Tools and languages like Modelica [13], Simulink [17], UML [18], and SysML [19] are used for modeling both the system and the software aspects, but combining them for joint usage is still a big challenge [20], [21].

Many approaches have been proposed to integrate software engineering with systems engineering and to add computations to conceptual models. These approaches divide the system under consideration into domains, where each domain is modeled separately. OMG has issued a request for proposals to SysML, v2 RFP [22], which is meant to replace the current SysML [19]. The new proposed features emphasize that the current SysML does not have computational and execution capabilities. Indeed, the current SysML parametric diagram, which is the only diagram meant to handle computations, is static and not executable. OMG has tried to integrate conceptual modeling with computational and execution capabilities by developing xUML [23] and fUML [18], [24], [25], [26]. However, these solutions require to divide the system into several subsystems that have little or no interaction between them. The acute need to integrate conceptual modeling with computational and execution capabilities is clearly demonstrated in our prior work [7], in which we could not seamlessly extend the conceptual model to include the computations.

The common wisdom is that modeling and simulation are valuable solutions for system analysis and improve the development of complex cyber-physical systems [27]. Another approach for creating an executable model has shown the limitations of system simulation and execution [28]. The simulation lacks information about the main purpose of the system, and achieving a model that is amenable to simulation or execution requires four steps: (1) starting with the abstract "target system", (2) translating it to an "abstract model" that consists of formal definitions based on the target model (3) building a "computational model" that represents the abstract model and is founded on formal rules and definitions of agents, and (4) "software model", which is translation of the computational model. Having performed these steps, we end up with four models, but we execute only the last one. Although the final model is executable, it might not be consistent with the original purpose, as in each step some piece of information may be lost or changed [28]. The current research is different in that rather than four models, we keep evolving the same original abstract model, avoiding loss of intent and information.

3. OPM: A Brief Review

In this work we use OPM [4], [5] ISO 19450:2015 and its MAXIM extension [29] for computations. OPM is a systems modeling approach [30] that represents the function, structure and behavior of any system using only two kinds of things: objects—things that exist, and processes—things that transform objects (Figure 1).

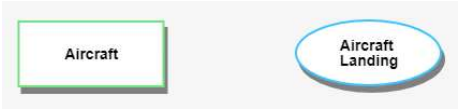

Figure 1. Example of an OPM object (left) and process (right)

Any OPM thing can be informatical (not shaded) or physical (shaded). An informatical object can be computational, in which case it potentially has a value, unit of measurement, and an alias—a short, space-less name for use in formulas, as exemplified in Figure 2.

Figure 2. Left: an OPM physical object that represents an **Aircraft**[2]*; Right:an informatical and computational object that represents a* **Weight** *of 71000 kg with the alias* **W**

Figure 3. Left: the OPM physical process **Aircraft Landing***; Right: the computational process* **Main Static Force Calculating***, which gets four computational objects with aliases* **A**, **W**, **N** *and* **D** *(not shown), and returns the* **Main Static Force** *object (not shown). The code of the computational process on the right appears in OPCloud while hovering over the process.*

An informatical process can also be computational, in which case it accepts as input parameters one or more object values and performs a specified computation to produce a resulting object value. The difference between a physical process and an informatical and computational one is shown in Figure 3.

OPM things are connected by links, which graphically express relations (Table 1). Any OPM model consists of two parts: (1) the OPD set: a set of Object Process Diagrams (OPDs) and, (2) the OPL spec: a collection of sentences in a subset of English (or any other natural language), called Object Process Language (OPL). Each OPD construct—things connected by links—is reflected textually as one or more OPL sentences.

An OPM model can be presented at various levels of detail in different, interconnected views, each being an OPD. The top-level OPD is called System Diagram (SD), which usually consists of one systemic process and its operand – the object which that process transforms. Together, the process and the operand are the function of the system. As SD is just a bird's-eye view of the system, each OPD can be refined in one of several ways in order to expose deeper levels of detail. In this work, we use only process in-zooming to show subprocesses, of which the main process consists, and their temporal (sequential or parallel) execution order, based on their vertical position, where higher processes happen before lower ones. MAXIM is developed and implemented as part of OPCloud [31], a collaborative cloud-based software environment for OPM-based modeling.

Table 1: Example of the main procedural links in OPM

Link name	Example
Result	Creating → File
Consumption	Eating ← Food
Effect	Pedal Pressing ↔ Speed
Agent	Eating ● Person
Instrument	Eating ○ Fork
Invocation	Test Finishing → Test Submitting

4. THE OPM CONCEPTUAL MODEL

In this section, we explain the conceptual Vee model. The landing gear is the part of the aircraft designed to enable its takeoff and landing. The latter requires absorbing great dynamic loads. A typical landing gear has three parts: one nose landing gear and two main landing gears. Each landing gear consists of a set of two or more tires and an oleo-pneumatic shock absorber strut. During aircraft landing, some of the loads are transmitted to the airframe through the landing gear. By determining the dynamic landing loads correctly, the landing gear weight can be reduced, enabling better aircraft performance.

What we have found after constructing the conceptual model was that the modeling process is also instrumental in finding in the free text document incomplete information and possibly contradicting requirements or wrong, unfounded basic assumptions.

We start with creating a high-level conceptual Vee model [7], [9], [10], [32] of the entire aircraft, which is circled by a solid black line on the left side of Figure 4. We then focus on the **Cargo Transporting** process and the objects involved in it, circled by the dashed black line in Figure 4. Next, based on our previous work [7], we will gradually shift to modeling the landing gear. We first refine each of the processes in the OPM Vee model in Figure 4 by zooming into it. According to [7], the **Aircraft Realizing** process is refined into three subprocesses, shown in Figure 5: **Aircraft Level Integrating**, **Aircraft Level Requirements Verifying**, and **Aircraft Certifying**. The object **Aircraft** has several states, two of which are visible: its initial state, **integrated**, and its final state, **certified**. This is expressed by the OPL sentences in Figure 6, which are generated automatically by OPCloud in response to the modeler's graphic input.

[2] Names of OPM entities (objects, processes, and states) in OPM models are in **bold Arial font.**

The result of the **Aircraft Level Integrating** process is an **integrated Aircraft**. In order for the second subprocess, **Aircraft Level Requirements Verifying**, to start executing, **Aircraft** has to be at state **integrated**. This requirement is expressed by the instrument link from this state to **Aircraft Level Requirements Verifying**. **Aircraft Certifying** changes the state of **Aircraft** from **integrated** to **certified**. A new input regarding the aircraft is known at this stage – the **Aircraft** is built according to **Aircraft Design**.

In our previous work [7], another, separate model was built in parallel to the Vee model for describing the **Cargo Transporting** process, circled in dashed black in Figure 4. The purpose of that model was to describe the market need of a potential customer, identified as the object **STKH Need 001** (where **STKH** is an abbreviation for stakeholder), whose state **airborne** payload **transportation** expresses the content of the need. This need is expressed in marketing requirement terms as the object **Marketing Req 001**, whose value is **airborne cargo transportation**. The **Aircraft** realizes the formal need, and both are instruments for the **Cargo Transporting** process, which changes the state of the object **Cargo** from **origin** to **destination**. **Cargo** exhibits the attribute **Weight** and consists of four parts: **Crew**, **Passenger**, **Baggage** and **Equipment**.

While the Vee model (Figure 4 left) and the **Cargo Transporting** model (Figure 4 right) were built as two separate models, the fact that they have a common object, **Aircraft** (and its parts), has enabled us to integrate them into a single model.

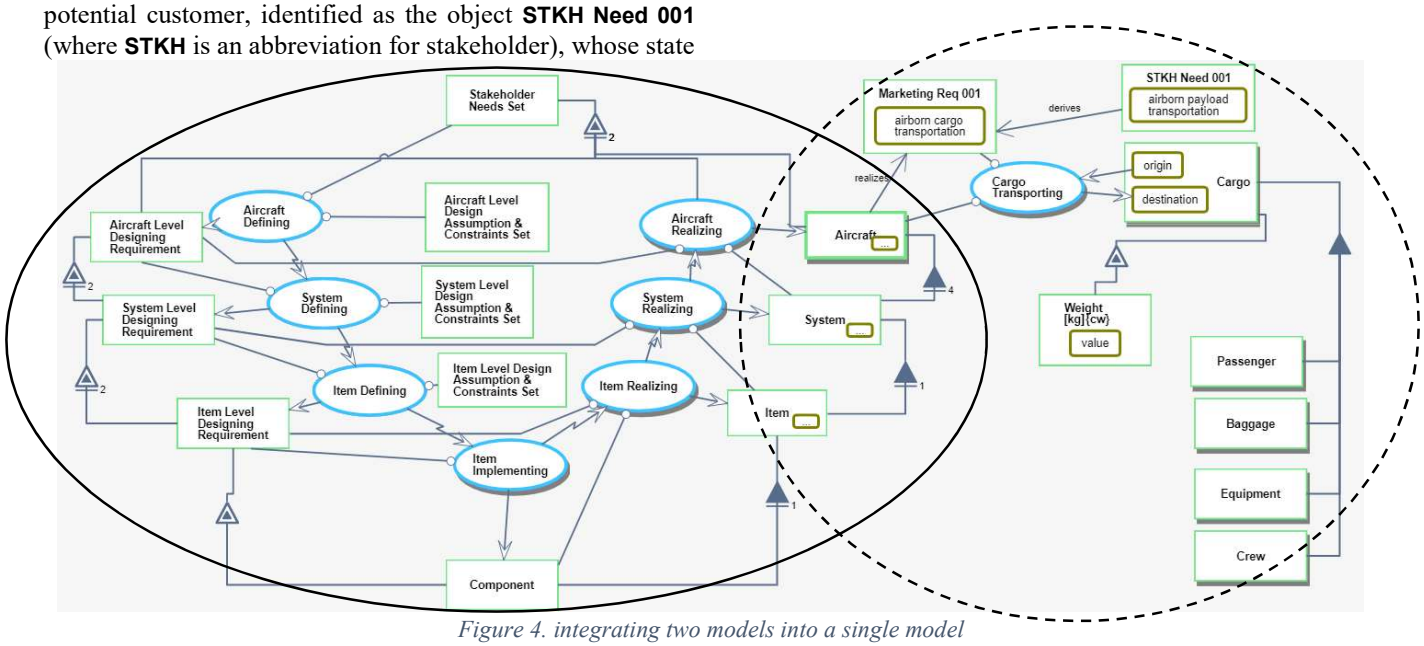

Figure 4. integrating two models into a single model

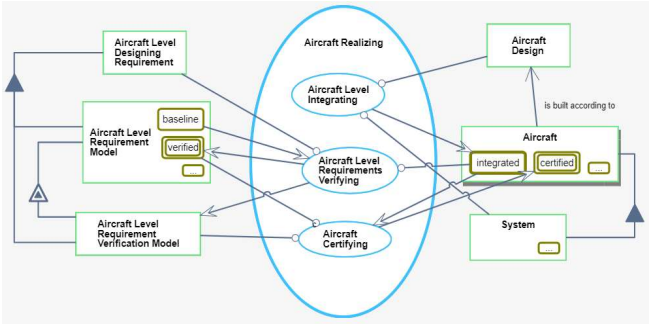

Figure 5. **Aircraft Realizing** *in-zoomed*

Aircraft can be integrated and certified.

State integrated of Aircraft is initial.

State certified of Aircraft is final.

Figure 6. OPL sentences describing the initial and final states of **Aircraft**

Descending several levels of detail down the OPD tree (the tree of increasingly more detailed OPDs) of our model into **Cargo Transporting**, we get to the most refined level presented in our previous work [7], which shows in Figure 7 the **Landing Shocks Absorbing** process in-zoomed. In this paper, we focus on the **Energy Storing** subprocess of this process.

The **Energy Storing** process changes the state of the **Tire** from **inflated** to **compressed**. The instruments of this process are **Nose Tire** and **Main Tire**, which are parts of **Tire**, as expressed by the OPM aggregation-participation links from **Tire** to **Nose Tire** and **Main Tire**. As explained in Section 5, we later realized that the aggregation-participation relation between **Tire** on one hand and **Nose Tire** and **Main Tire** on the other hand were incorrect, because the **Nose Tire** and the **Main Tire** are specializations of **Tire**, not parts of it, so a generalization-specialization relation (which gives rise to inheritance) should be used. Moreover, we modeled **Tire** as an object that exhibits five attributes: **Nominal Diameter**, **Wheel Nominal Diameter**, **Load Rating**, **Stiffness Coefficient**, and **Tire Pneumatic Spring Force**. In Section 5 we explain that this part of the model is problematic as well. Finally, The **Tire** and the **Oleo-Pneumatic Shock Absorber Strut** are both parts of the **Landing Gear**. The model contains additional objects and their attributes, but considering the main purpose of our paper, we focus on the ones specified above.

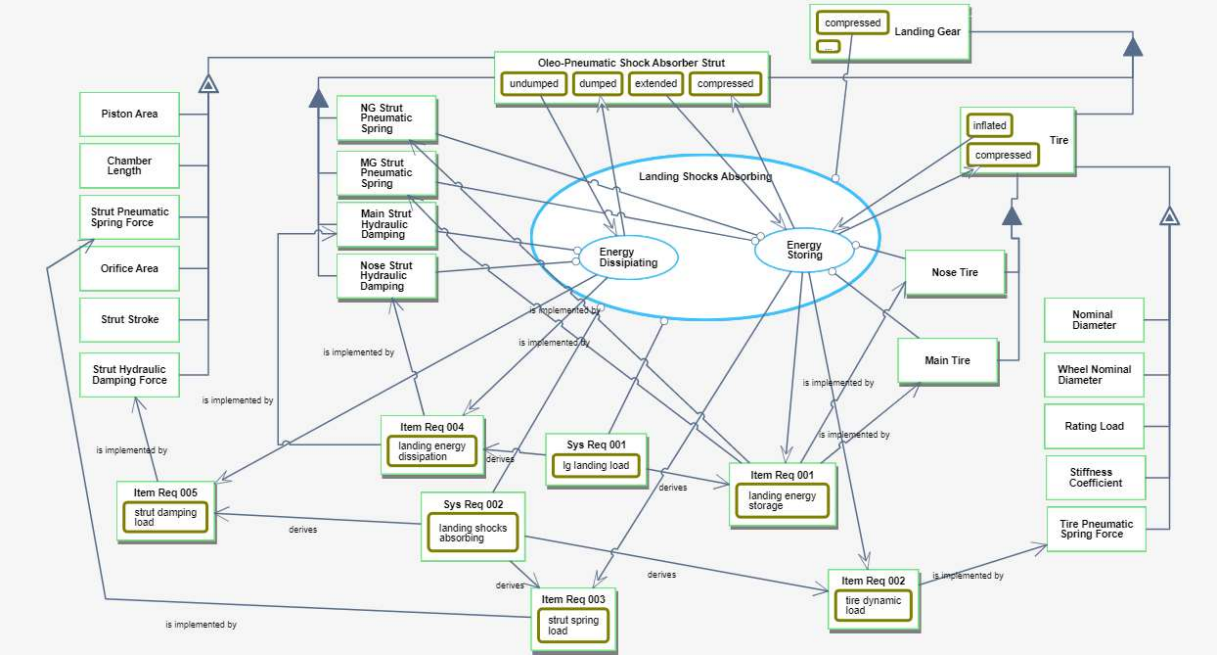

Figure 7. The **Landing Shocks Absorbing** *process in-zoomed*

5. How Did the Computational Model Extension Help Detect Errors?

In this section, we continue building our model by adding to it the computational part, including adding values to attributes and executable or computable functions as code to leaf processes. Using our previous work [7] as a starting point, the goal in this work was to calculate the static load on the aircraft landing gear's tire, considering the aircraft and its landing gear parameters in a single model. The calculated parameters are needed for selecting the most suitable tires. Having selected the tire type, the tires' reaction forces can be calculated and used further to determine constraints on the struts. These, in turn, are needed to determine the maximal allowed cargo weight.

Since the formulas were already known and we modeled all the required parameters (see Figure 7), initially, this calculation task seemed to be relatively easy. However, when we started to assign values to computational objects and functions to computational processes in our model, we realized that the model was neither accurate nor sufficiently detailed, and we had to correct and refine it. Our experience has been that following the addition of computational capabilities, logical errors and inaccuracies in the conceptual model are discovered, because as numerical values or parameters that are used in equations are added, there is no room for vague or incomplete definition of model parameters. In what follows, we describe examples of errors that we found out and changes we had to make in the model in order to make it computable and executable. These unfolding revelations triggered the idea of the model fidelity hierarchy, which became the higher-level focus of this research.

5.1 Static Load and Load Rating Calculation

Our first goal was to calculate the static load on the landing gears. To do this, we use two different equations [7]: one for the nose landing gear (Equation 1) and the other for the two main landing gears (Equation 2), where D is the **Wheel Base**, A is the center of gravity position, and W is the **Weight** of the aircraft. For safety, the maximum weight before flight (with fuel) is taken as W =71,000 kg.

$$F_{S,N} = \frac{D - A}{D} W$$

Equation 1. The ***static load*** *of the nose landing gear.*

$$F_{S,N} = N \frac{A}{D} W$$

Equation 2. The ***static load*** *of the main landing gear.*

After understanding the required calculations, we realized that there is a need to compute two different static loads: one for the two (identical) main landing gear and another for the nose landing gear. As the static load is an attribute of **Landing Gear**, we needed to add two objects to represent the two kinds of landing gear: **Main Landing Gear** and **Nose Landing Gear**. As noted, in our previous work [7] we performed the calculations in Matlab, separately from the conceptual model. Therefore, the absence of these two objects was not noticed. The integration of the computations into the conceptual model was not possible prior to conceiving how conceptual and computational modeling can be integrated into, and performed by, the same object-process modeling framework. The idea for doing this was to treat mathematical functions used for computations as informatical processes, and the input and output parameters of these functions as (informatical) objects, which are attributes of physical objects in the engineering problem. Further, this underlying integration idea needed to be implemented in the OPCloud software

environment before we could use it in our Landing Gear problem as a case in point. We started refining the conceptual model part, presented in Figure 7, by adding these objects and connecting them to **Landing Gear** by a generalization-specialization link (white triangle), as shown in Figure 8. After adding the objects **Main Landing Gear** and **Nose Landing Gear**, we realized that the objects **Main Tire** and **Nose Tire** should be parts of the **Main Landing Gear** and the **Nose Landing Gear**, respectively, so in Figure 8 we added two aggregation participation links: one from **Main Landing Gear** to **Main Tire** and another one from **Nose Landing Gear** to **Nose Tire**. Finally, we changed the aggregation-participation links from **Tire** to **Main Tire** and **Nose Tire** to be generalization-specialization links, since the main and nose tires are kinds, not parts, of **Tire**.

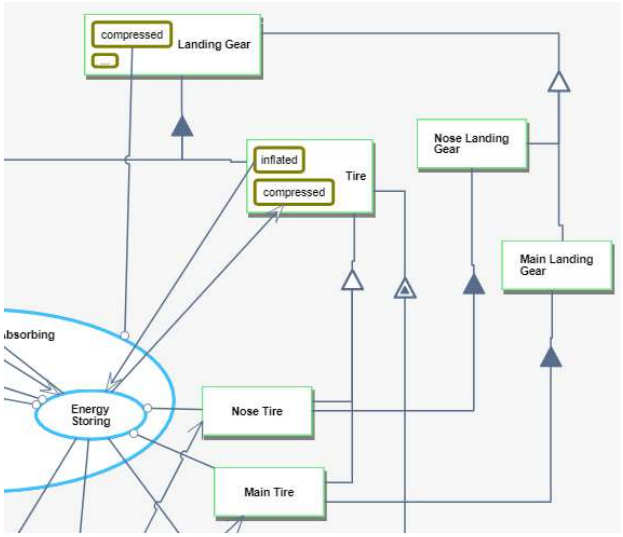

Figure 8. The fixed part of Figure 7, where we added **Main Landing Gear** *and* **Nose Landing Gear** *as specializations of* **Landing Gear**, *and refined the links*

Having made these changes, we could create a refined OPD by zooming into the **Energy Storing** process and adding the first subprocess, **Landing Gear Parameters Defining**, which was supposed to calculate the static load. We then zoomed into that process and started to get the OPD in Figure 9. For the **Nose Static Force Calculating ()**, where the **()** symbol signifies a computational process, we used **Fwd CG Position** with the alias **A**, unit **m** (meter), and value **5**, **Wheel Base** with alias **D**, unit **m** and value **5**, and **Weight** of **Aircraft** measured in **kg**, with the alias **W** and value **71000**. The result of this process will be assigned as the value of the object **Nose Static Load** with units **kg** and alias **Fsn**. For the **Main Static Force Calculating ()**, we use **Aft CG Position** with the alias **A**, unit **m** and value **5.4**, **MG Strut Quantity** with alias **N**, which is unitless and has the value **2**, and with the same **Weight** of **Aircraft** and **Wheel Base** used for **Nose Static Force Calculating ()**. The result of this process will be assigned as the value of the object **Main Static Load** with units **kg** and alias **Fsm**.

As the first subprocess, **Landing Gear Parameters Defining** is completely modeled and we have the results of **Nose Static Load** and **Main Static Load** (the main and nose static loads, respectively), we can now add the second subprocess, **Tires Selecting**. The purpose of this subprocess is to calculate the load rating of each tire, and based on the computed tire load rating results, the appropriate tires can be selected for the nose and main landing gears. As shown in Figure 10, the **Load Rating** F_{Sti} is a function of the static load, and although it is calculated by using the same formula (Equation 3), the **Load Rating** of the **Main Tire** is different from that of the **Nose Tire**, because the static load input F_S for the nose landing gear is different than that for the main landing gear.

$$F_{\mathrm{Sti}} = 1.07 * \frac{F_S}{2}$$

Equation 3: The ***load rating*** *formula.* ***Fs*** *is the* ***static load*** *(****nose*** *or* ***main****)*

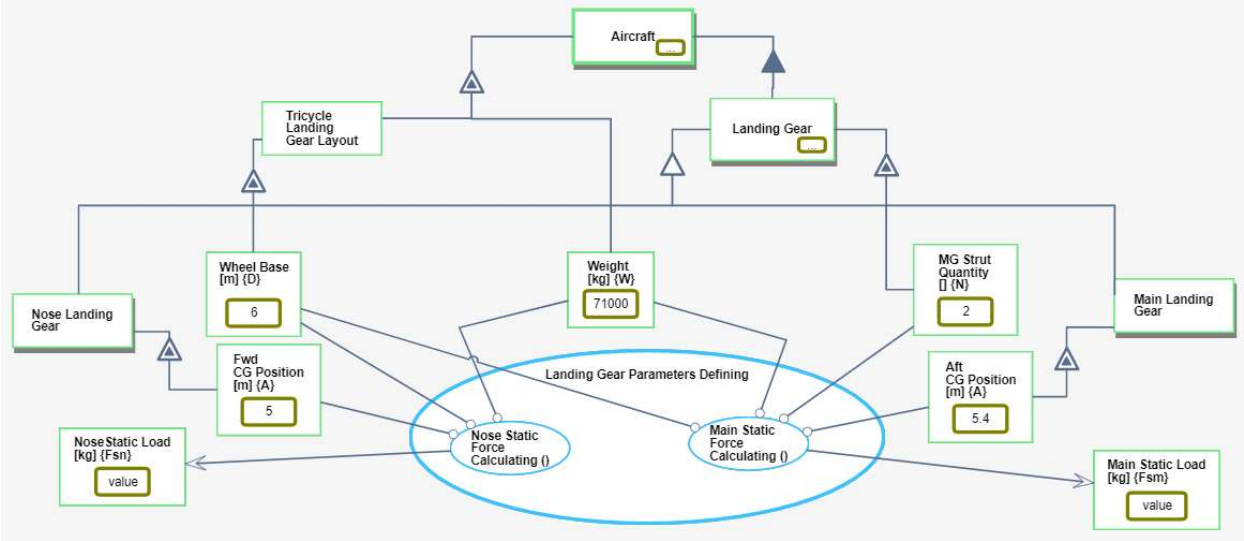

Figure 9. **Landing Gear Parameters Defining** *in-zoom for calculating the* **Static Load** *for each landing gear*

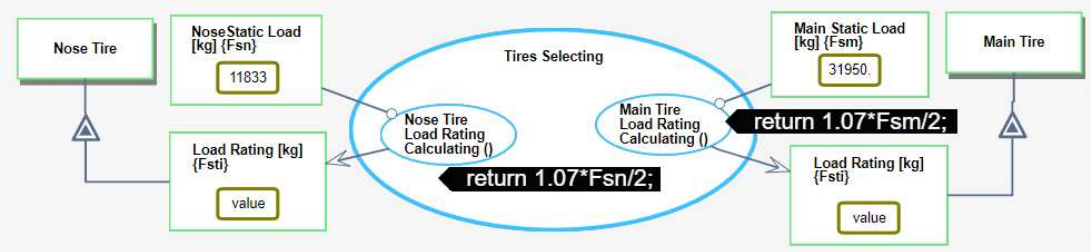

Figure 10. **Tires Selecting** *in-zoom for calculating the required* **Load Rating** *for tires*

5.2 TIRE PARAMETERS DEFINING

The third and last process of the in-zoomed **Energy Storing** process is **Tire Parameters Defining**. A computational process is a leaf process; it cannot be further refined. As shown in Figure 11, **Tire Parameters Defining** is an informatical process, but not a computational one, as it is further refined into four computational (and hence leaf-level) subprocesses: **Nose Tire Stiffness Coefficient Calculating ()**, **Main Tire Stiffness Coefficient Calculating ()**, **Nose Reaction Force Calculating ()**, and **Main Reaction Force Calculating ()**.

Calculating the stiffness coefficients requires the wheel and tire nominal diameters and load ratings, which are different for the nose and main landing gears, as shown in Equation 4, where F_{Sti} is the **Load Rating** of the **Tire**, dw is **Wheel Nominal Diameter** and dti is **Tire Nominal Diameter**..

$$K_{ti} = F_{Sti} * \left[\frac{1}{6} * \left(1 - \frac{dw}{dti}\right)\right]^{-1}$$

Equation 4: The formula for calculating the **Stiffness Coefficient**

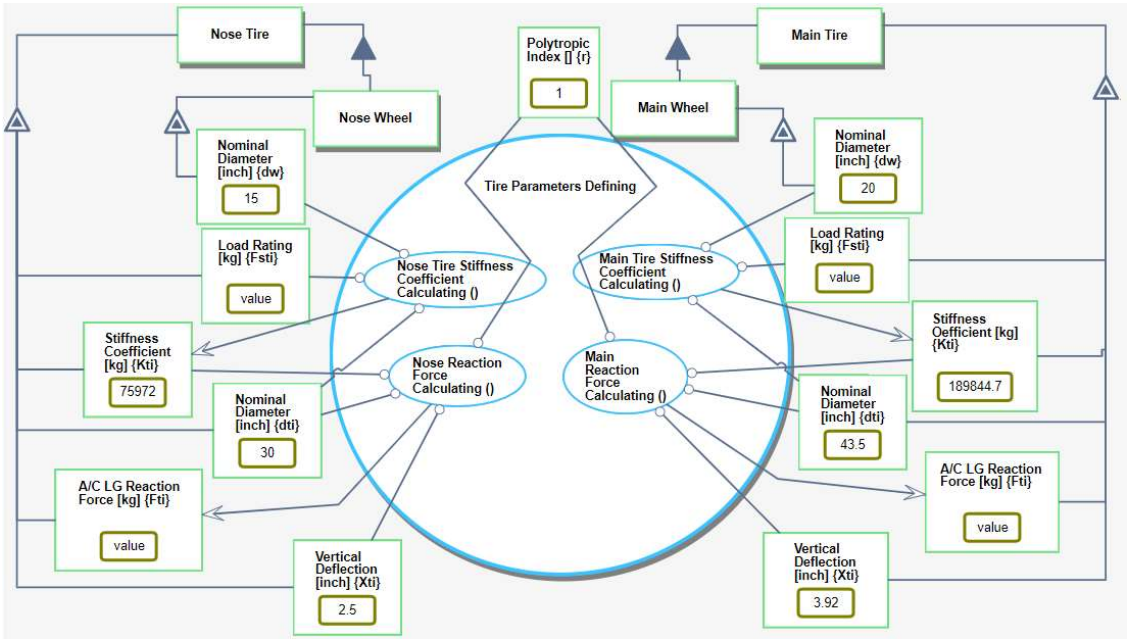

Figure 11. **Tire Parameters Defining** *in-zoomed*

Therefore, when we started to model the process **Stiffness Coefficient Calculating ()**, we realized that our conceptual model has a major flaw! According to Figure 7, all the five attributes of the tire—**Nominal Diameter**, **Wheel Nominal Diameter**, **Stiffness Coefficient**, **Load Rating**, and **Tire Pneumatic Spring Force**—must not be connected directly to the object **Tire**. Rather, these five **Main Tire** attributes are different than the **Nose Tire** ones, and therefore in the model they must be separated too! Following this new insight, we went back and fixed the conceptual model part by deleting these five tire attributes from the OPD in Figure 7 and adding them correctly twice, once for **Nose Tire** and once for **Main Tire**, as shown in Figure 11.

6. THE OPM MODEL EXECUTION

So far, we accounted for modeling errors that were detected thanks to the need to specify the precise calculations and integrate them into the conceptual model. These errors were found while adding to the purely conceptual model the computational things (objects and processes) needed for calculating the landing gear parameters. At this point, one might think that the model is complete, because it has everything it takes to compute the model parameters. Moreover, the model also seems to be correct, because it contains the proper equations provided by the expert aeronautics engineer. However, it turns out that even at this advanced stage, the model contained errors, albeit of a

different nature, which escaped being detected. These errors could only be found and corrected during execution, as they stem from wrong values given initially to the parameters, as we describe next.

Having completed the model and prepared it for calculations of the various landing gear parameters, which are done during the model execution, we assigned the parameter values to all the input objects by typing them directly into the value slots in the OPM model using OPCloud, and clicked to execute the model in order to get the desired results. As the execution was running, we suddenly saw that while the **Main Static Load** is calculated, providing reasonable result, the **Nose Static Load** is zero (see bottom left in Figure 12), which makes no sense.

We quickly found out that both **Wheel Base {D}** and **Fwd CG Position {A}** were assigned the value **5**. The formula for calculating the **Nose Static Load** is shown in Equation 1, and when the model subtracts **Fwd CG Position {A}** from **Wheel Base {D}**, the result is zero. The value of **Wheel Base {D}** had to be **6**. With the correct **Wheel Base** value, the execution ran smoothly, yielding the desired parameter results of the nose and main landing gears and tires, as shown in Figure 14.

Our takeaway lesson from this incident is that just as integrating computations into the conceptual model helps us find errors in the conceptual model, model execution helps us find errors in computations – in this case it was the initialization of the object **Wheel Base**.

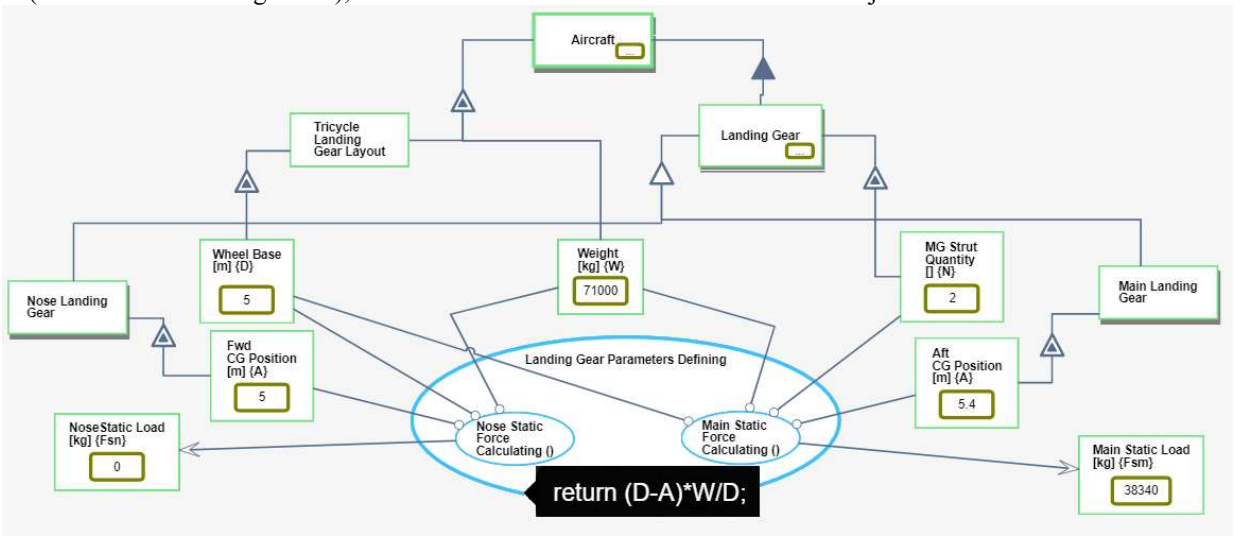

Figure 12. The value of **Main Static Load {Fsm}** *is 38340 but the value of* **Nose Static Load {Fsn}** *is 0*

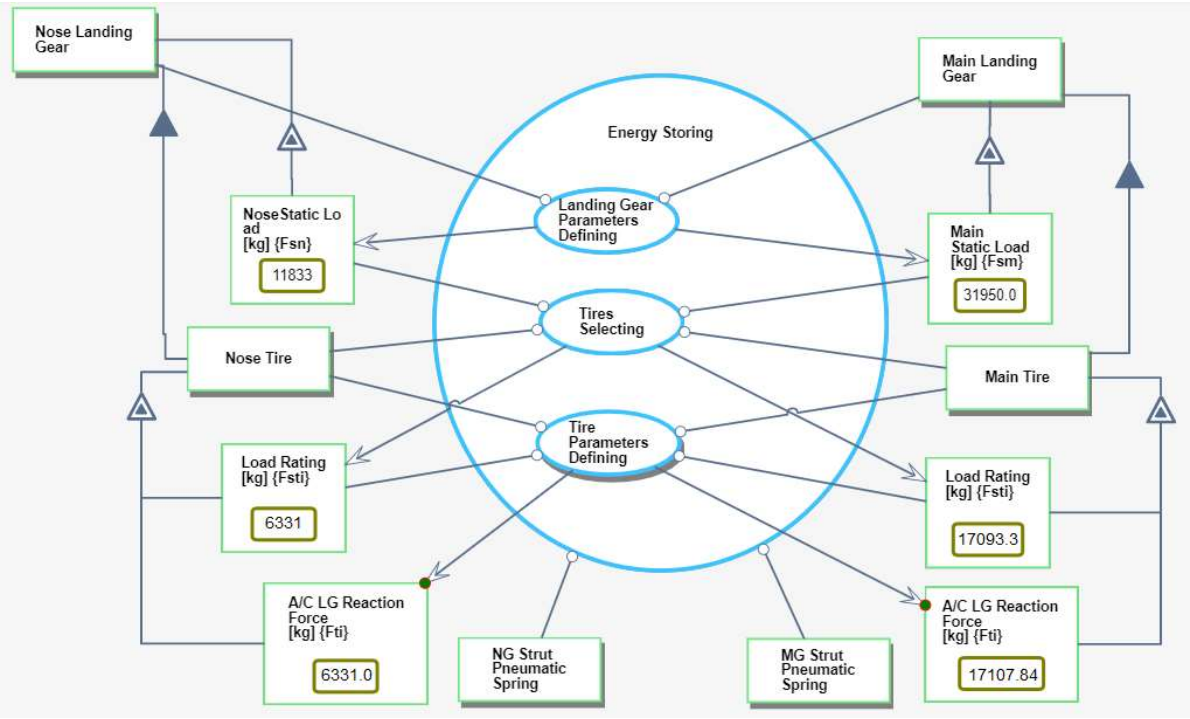

Figure 13. calculated parameters of the **nose** *and* **main landing gears** *and* **tires**

7. SUMMARY AND DISCUSSION

As we were augmenting the conceptual model of an aircraft landing gear with computational objects and processes in order to obtain actual parameter values of a given set of input parameters, we came across the realization that there is a model fidelity hierarchy, which progresses from spoken to written text, and from there to conceptual and computational models, culminating in the execution of an executable model. With each transition from one level in this model fidelity hierarchy to the next, the accuracy requirements increase, causing a new set of errors to be detected and corrected. Importantly, the layers in our evolving model are not clearly distinguishable. Rather, as they all use the same underlying object-process paradigm and symbol set, they blend seamlessly with each other with no clear-cut borders. This unified framework avoids the loss of often critical information associated with transitioning from one modeling framework to the next.

Conceptual modeling supports the development of the qualitative aspect—what is the benefit of the systems to its beneficiaries and other stakeholders, what is the system's main function—the main process and the operand(s) which that process transforms to provide the benefit, what are the subprocesses of the main process and the objects that they require or transform, what are the environmental objects and processes and how they interact with the system, etc. Following the Vee model, at some point during the conceptual modeling process, the system architects and engineers deem the model sufficiently detailed to be handed over to the various vertical engineering domain experts—aerospace, mechanical, electrical, and software engineers. They are expected to "take the baton" and carry it over by designing the various components using different engineering tools they are familiar with and accustomed to, while relying on the conceptual model as being the ultimate source of authority. It is at this stage that serious computational considerations are made and operations take place. Our study has demonstrated that there is important synergy between system-level conceptual qualitative modeling and component- and discipline-level design, where more concrete and quantitative considerations come to play. Adding the quantitative layer on top of the qualitative one helps in discovering design errors that would otherwise be either detected at a later stage in the development process of the system, incurring significant costs and project delays, or worse yet, escape detection and remain uncorrected in the system, causing even worse, sometimes detrimental or fatal damage during the system's use and service. This approach is information lossless because we keep building and adding details on top of the current evolving model. The execution layer on top of both the qualitative and quantitative modeling helps in further identifying problems in the system. Specifically, we have described two transitions in the modeling process that not only make it more grounded and concrete, but, not less importantly, are most instrumental in revealing errors in the system's conceptual model. The first transition is from conceptual, qualitative to computational, quantitative modeling, during which we seamlessly incorporate into the conceptual model executable code of mathematical equations that express the underlying physics and engineering considerations. The code, embedded into leaf-level processes in the model, can be of any complexity level, and include not just mathematical formulae, but also any control flows such as conditions and loops. This first kind of transition, from conceptual to computational model, revealed conceptual modeling errors and triggered changes in the model.

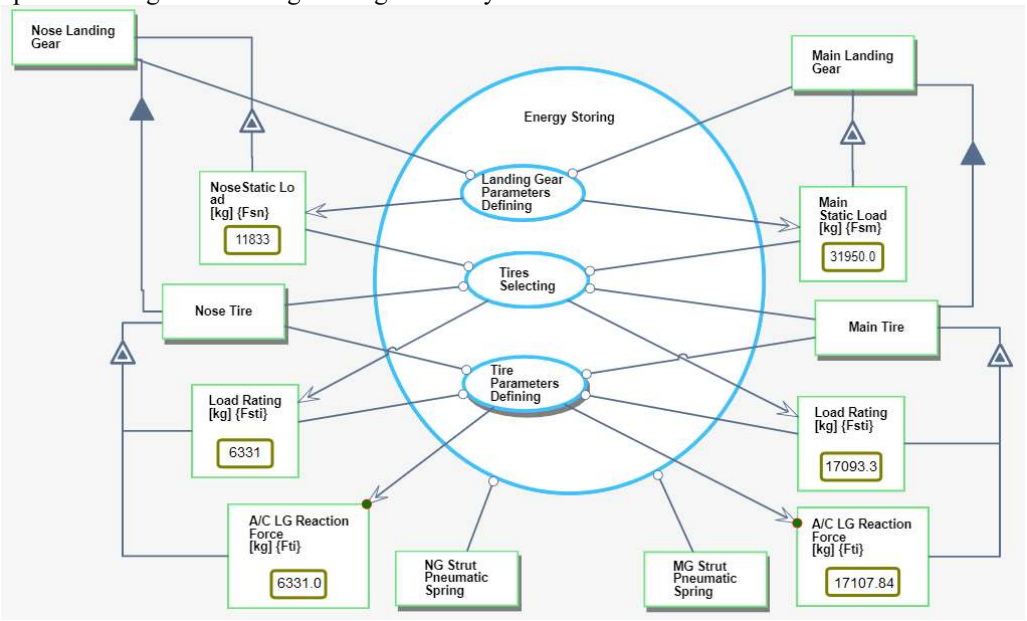

Figure 14. calculated parameters of the **nose** *and* **main landing gears** *and* **tires**

The second transition is moving from the combined qualitative-quantitative model to running the execution of that model with the objective of calculating system parameter values. When actual values are assigned to input objects, the

computation is performed by executing the model, yielding sought parameter values. Errors found at this stage can be not just wrong inputs, as was the case in our research, but also flaws in the formulae or even errors that were caused earlier on, during the conceptual modeling stage. The ability to spot and correct errors as early as possible in the system's lifecycle is of paramount importance, since the cost of correcting errors grows exponentially with the system lifecycle stage [33]. Two important implications of our work are the following.

(1) The model fidelity hierarchy of systems, which emerged as an unintended consequence of our research, has four levels: (a) Ideation and informal idea communicating – At the most abstract and fuzzy level there are the processes of ideation, verbal and written text specification. (b) Conceptual-qualitative modeling – The model of the system resulting from this process expresses the specification formally. In our case, this model was created using OPCloud, which applies OPM ISO 19450. (c) Computational augmenting and enhancing – The conceptual-qualitative model is augmented and enhanced with quantitative computational elements. In our case, these were integrated seamlessly into the conceptual model using MAXIM on the same OPCloud platform. (d) Model executing – executing the model to ascertain that the conceptual-qualitative-and-quantitative model runs as expected. Different kinds of mistakes are discovered and corrected while transitioning from one level to the next, which were not discovered at the earlier level despite intensive checks and best efforts, because they are so subtle that only the fidelity required at the next level exposes them. Figure 15 is an OPD of the Model Fidelity Hierarchy, which graphically summarizes our main findings.

(2) Modeling a complex system, including both its conceptual-qualitative and computational-quantitative parts, using the same modeling paradigm, has the advantage of a streamlined, lossless process. The usage of OPM has made it possible to reveal the model fidelity hierarchy and enabled level transitions to be information lossless, providing the most value while requiring the minimal effort.

The value added by the additional modeling layers with their computation and simulation capabilities is that they enable the creation of a seamless continuum, spanning the spectrum that starts with the abstract level of business requirements and goes all the way to specialized disciplinary engineering considerations.

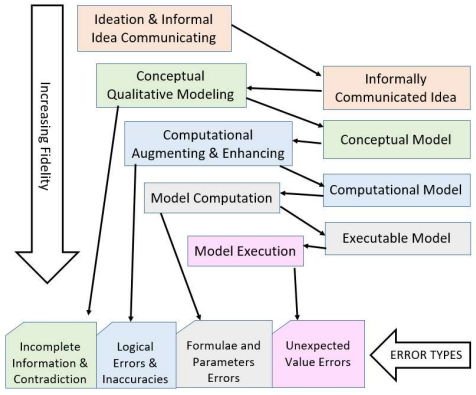

Figure 15. Model Fidelity Hierarchy diagram

8. LIMITATION AND FUTURE RESEARCH

The limitation of this research is that detection of model errors was not the original purpose of the research. Rather, the model fidelity hierarchy was discovered unintentionally. Therefore, research that is dedicated specifically to the model fidelity hierarchy may deepen our understanding of all the hierarchy levels and better characterize the kinds of errors revealed with each transition from one level to the next, as well as the need to backtrack to correct errors in previous layers. Moreover, a bottom-up modeling approach might provide additional insights.

As future research, our plans are in two directions: extending the model fidelity hierarchy and improving the OPM modeling capabilities. As part of the former direction, we have extended the hierarchy to include a fifth level – Hardware in the Loop [34], where we combine OPM and its MAXIM extension to execute models to which hardware, including robots, is connected, getting input from sensors and commanding actuators to respond in real time. In addition, we plan to further characterize the various errors discovered at each fidelity level and apply the findings in OPCloud to improve the modeling process. As for extending OPM, we suggest two actions: (1) Implementing the generalization-specialization and the inheritance it induces, as specified in [3], [4]. This will eliminate the need to model separate sets of attributes for specialized objects, such as the nose and main landing gears in our case study, while still being able to input and get separate parameter values for each set. Enabling this option might involve also the OPM path mechanism. (2) Enable automated batch execution, in which a massive amount of object input values will be fed from one of two sources: (a) an external source, such as a MS Excel file or a Gmail Sheet, and (b) synthetic input values, generated randomly from a probability distribution function that mimics the behavior of the real system. By automatically executing the model multiple times with diverse sets of values, we are much more likely to reveal additional wrongly modeled cases or errors in formulas defined as functions in computational process. Examples of such errors include division by zero and a software function having if-else conditions with input values that fit neither the if nor the else part of the condition, yielding an 'undefined' result value that leads to revealing the error.

ACKNOWLEDGEMENT

This research was partially funded by the Gordon Center for Systems Engineering at the Technion, Israel Institute for Technology.

REFERENCES

[1] D. Firesmith, "Modern Requirements Specification," *J. Object Technol.*, vol. 2, pp. 53–64, 2003.

[2] A. Blekhman, J. P. Wachs, D. Dori, and S. Member, "Model-Based System Specification With Tesperanto : Readable Text From Formal Graphics," pp. 1–11, 2015.

[3] J. Otero and W. Kintsch, "Failures to Detect

Contradictions in a Text: What Readers Believe versus What They Read," *Psychol. Sci.*, vol. 3, no. 4, pp. 229–235, 1992.

[4] D. Dori, *Object-process methodology: a holistic systems paradigm*, 1st ed. Verlag Berlin Heidelberg: Springer, 2002.

[5] D. Dori, *Model-based systems engineering with OPM and SysML*. 2016.

[6] K. Dori Dov, Hanan Kohen, Ahmad Jbara, Niva Wengrowicz, Rea Lavi, Natali Levi-Soskin and U. S. Bernstein, "OPCloud: The foundation of a new OPM-based system design paradigm for Industry 4," in *Systems engineering in the fourth industrial revolution: Big data, novel technologies, and modern systems engineering*, & A. Z. R. S. Kenett, R. S. Swarz, Ed. Wiley-Blackwell, 2019, pp. 243–271.

[7] L. Li, N. Levi-Soskin, A. Jbara, M. Karpe, and D. Dori, "Model-Based Systems Engineering for Aircraft Design with Dynamic Landing Constraints Using Object-Process Methodology," *IEEE Access*, vol. 7, pp. 1–1, 2019.

[8] "ISO / IEC JTC1 / SC7 N2683 PDTR Ballot PDTR 19760 Systems Engineering – Guide for ISO / IEC 15288 (System Life Cycle Processes)," 2002.

[9] K. Forsberg and H. Mooz, "The relationship of system engineering to the project cycle," in *INCOSE International Symposium*, 1991, vol. 1, no. 1, pp. 57–65.

[10] A.-P. Bröhl, *Das V-Modell: Der Standard für die Softwareentwicklung mit Praxisleitfaden*. Oldenbourg, 1993.

[11] A. Johanson and W. Hasselbring, "Software Engineering for Computational Science: Past, Present, Future," *Comput. Sci. Eng.*, vol. PP, no. 99, p. 1, 2018.

[12] A. Pyster *et al.*, "Exploring the Relationship between Systems Engineering and Software Engineering," *Procedia - Procedia Comput. Sci.*, vol. 44, pp. 708–717, 2015.

[13] M. Tiller, *Introduction to Physical Modeling with Modelica*, 1st ed., vol. 615. Springer US, 2001.

[14] P. Dobrev, B. S. Innovations, and G. Angelova, "From Conceptual Structures to Semantic Interoperability of Content," no. July 2007.

[15] J. A. Muguira, "The Levels of Conceptual Interoperability Model," no. September 2003.

[16] B. Kruse and M. Blackburn, "Collaborating with OpenMBEE as an Authoritative Source Truth Environment," *Procedia Comput. Sci.*, vol. 153, pp. 277–284, 2019.

[17] R. Klee, H., Allen, *Simulation of Dynamic Systems with MATLAB and Simulink*, 3rd ed. 2018.

[18] The Object Management Group, "Semantics of a Foundational Subset for Executable UML Models (fUML)," *Object Manag. Gr.*, no. October, p. 441, 2012.

[19] OMG group, *OMG, Systems Modeling Language (SYSML) Specification*, 1.3. 2012.

[20] "Design patterns for open tool integration," *Softw Syst Model*, vol. 4, pp. 157–170, 2005.

[21] B. Akesson, A. Molnos, A. Hansson, J. Ambrose Angelo, and K. Goossens, "Composability and Predictability for Independent Application Development, Verification, and Execution," in *Multiprocessor System-on-Chip*, M. Hübner and J. Becker, Eds. Springer Verlag, 2010, pp. 25–56.

[22] O. M. Group, "Systems Modeling Language (SysML®) v2 Request For Proposal (RFP)," 2018.

[23] S. J. Mellor and M. J. Balcer, "Executable and Translatable UML."

[24] E. Seidewitz, "UML with Meaning: Executable Modeling in Foundational UML and the Alf Action Language," in *Proceedings of the 2014 ACM SIGAda Annual Conference on High Integrity Language Technology*, New York: ACM, 2014, pp. 61--68.

[25] C. L. L. and I. L. and B. P. and S. M. and I. G. Czibula, "Using a fUML Action Language to Construct UML Models," in *2009 11th International Symposium on Symbolic and Numeric Algorithms for Scientific Computing*, pp. 93–101.

[26] A. B. Specification, "Action Language for Foundational UML (Alf) Concrete Syntax for a UML Action Language," *Int. Bus.*, no. October 2010, 2011.

[27] D. Gianni, A. D'Ambrogio, and A. Tolk, *Modeling and Simulation-Based Systems Engineering Handbook*. .

[28] M. L. Federici, S. Redaelli, and G. Vizzari, "Models, Abstractions and Phases in Multi-Agent Based Simulation," no. January 2006.

[29] N. Levi-Soskin, R. Shaoul, H. Kohen, A. Jbara, and D. Dori, "Model-Based Diagnosis with FTTell: Assessing the Potential for Pediatric Failure to Thrive (FTT) During the Perinatal Stage," in *Information Systems: Research, Development, Applications, Education*, 2019, pp. 37–47.

[30] "ISO/PAS 19450 Automation systems and integration -- Object-Process Methodology," 2015. [Online]. Available: https://www.iso.org/standard/62274.html.

[31] "OPCloud." [Online]. Available: https://www.opcloud.tech/.

[32] I. Graessler, J. Hentze, and T. Bruckmann, "V-models for interdisciplinary systems engineering," *Proc. Int. Des. Conf. Des.*, vol. 2, pp. 747–756, 2018.

[33] A. Nasa and J. Space, "Error Cost Escalation Through the Project Life Cycle," 2019.

[34] H. Kohen and D. Dori, "Incorporating Hardware-in-the-Loop Simulation into Object-Process Methodology," in *14th Annual IEEE International Systems Conference (SysCon2020)*, 2020.